

⇒ The inventor of Python
Guido van Rossum - 1991

http → hyper text Transfer protocol.
https → S means secure
It means set of rules to follow
while taking your data to server → http.

→ firstprogram.py

①✓

print ("Hello World")

Hello World

print (7)

7

print ("byy")

byy

print (100 * 100)

10000

print (100 + 100)

200

Hello Ramesh

Ramesh

Ramesh

① → Comments, Escape Sequence, print Statement.

②✓

In Python ^{we} will write comments in single line AND Multi line ALSO
Comments ↓

This is a web page

This is not a web page ## But This is a page

Python ignores ## This and line would be comment.

⇒ Multi line Comments

You will put """ ~~RAMESH~~
GEHLOT
RAMESH
"""

OR

'''
GEHLOT
RAMESH
GEHLOT
'''

You wanna put double coma or single comma ³ the Three Time in 1 And Last

\n → Use for new line in one print Statement

P(- \n -)

-

-

Program →

```
"""
Hey SAM, Please dont remove this line
Author: RAMESH
Python 100v
"""
print ("Hey I am 'a' good boy" \n and this viewer is also a good boy/girl")
# This is a web Page
### This is not a web page # But this is a page
print ("Hey", 6, 7, sep = "~", end = "100\n")
print ("Samual")
```

→ Hey I Am a "good boy"
and this viewer is also a good boy/girl
Hey~6~7100
Samual

③ V ⇒ Variables And Data Types

What is variable?

Variable is like a container that holds data. Very similar to how our container in Kitchen holds sugar, salt etc. Creating a variable is like creating a placeholder in memory and assigning it some value. In python its as easy as writing:

different Data type

a = 1 → int

b = True → Yes or No Condition Boolean data:

c = "Harry" → String

d = None

1) Numeric data : int, float, complex

- int 3, -8, 0

- float 7.349, 9.0.000001

- complex 6+2i

2) Text data str:

str: "Hello World!!!", "Python Programming"

4(V)

Arithmetic operators

Operators	Oper Nam	Exmp
+	Addition	15+7
-	Subtraction	15-7
*	Multiplication	5*7
/	Exponential	15/7
%	Module	15%7
//	Floor Division	15//7

Python has different types of operators for different operations. To create a calculator we require arithmetic operators.

```

x, y = 3, 2
print (x+y)           # Output
print (x-y)           # = 5
print (x*y)           # = 6
print (x/y)           # = 1.5
print (x//y)          # = 1
print (x%y)           # = 1
print (-x)            # = -3
print (abs(-x))       # = 3
print (abs(-x))       # = 3
print (int(3.9))      # = 3
print (float(x))      # = 3.0
print (x**y)          # = 9

```


Calculator.py

a = 250

b = 180

```
print ("The value of", "a", "+", "b", "is:", a+b)
print ("The value of", "a", "-", "b", "is:", a-b)
print ("The value of", "a", "*", "b", "is:", a*b)
print ("The value of", "a", "/", "b", "is:", a/b)
print ("The value of", "a", "%", "b", "is:", a%b)
```

Output → The value of a+b is: 430

- - - a-b → 70

- - - a*b → 45000

- - - a/b → 1.3888888

- - - a%b → 70

* Typecasting in Python

The conversion of one data type into the other data type is known as type casting in python or type conversion in python.

Python Supports a wide variety of functions or methods like: int(), float(), str(), ord(), hex(), oct(), tuple(), set(), list(), dict(), etc. for the type casting in python.

Two types of Typecasting

1. Explicit Conversion (Explicit type casting in python)
2. Implicit Conversion (Implicit type casting in python)

Explicit typecasting:

The conversion of one data type into another data type, done via developer or programmers intervention or manually as per the requirements is known as Explicit type conversion.

It can be achieved with the help of Python's built-in type conversion function such as `int()`, `float()`, `hex()`, `oct()`, `str()`, etc.

Example of explicit typecasting

```
string = "15"
```

```
number = 7
```

```
string-number = int(string) # throws an error if the string
is not a valid integer
```

```
sum = number + string-number
```

```
print ("The Sum of both the number is:", sum)
```

Output: The Sum of both the number is 22.

Implicit type casting

Data types in Python do not have the same level i.e. ordering of data types is not the same of the data type have higher-order, and some have lower order. While performing any operation on variables with different data types in Python, one of the variable's data types will be changed to the higher data type. According to the level, one data type is converted into other by the Python interpreter itself (automatically). This is called implicit typecasting in Python.

Python converts a smaller data type to a higher data type to prevent data loss.

Exmp → # Python automatically converts

a to int

a = 7

print (type (a))

Python automatically converts b to float

b = 3.0

print (type (b))

Python automatically converts c to float as it is a float addition

c = a + b

print (c)

print (type (c))

Output: <class 'int'>

<class 'float'>

10.0

<class 'float'>

Taking user input in python

In python, we can take user input directly by using input() function. This input function gives a return value as string/character hence we have to pass that into a variable.

Syntax:

Variable = input()

But input function returns the value as string. Hence we have to typecast them whenever required to another datatype.

Exm

```
variable = int(input())
```

```
variable = float(input())
```

We can also display a text using input function. This will make input () function take user input and display a message as well

```
Exmp a = input("Enter the name:")  
print(a)
```

Output: Enter the name: Harry
Harry

What are strings?

In python, anything that you enclose between single or double quotation marks is considered a strings. A string is essentially a sequence or array of textual data. Strings are used when working with Unicode characters.

```
Example: name = "Harry"  
print("Hello," + name)
```

0 → Hello, Harry

Note: It does not matter whether you enclose your string in single or double quotes, the output remains the same.

Sometimes, the user might need to put quotation marks in between the strings. Example, consider the sentence: He said, "I want to eat an apple".

How will you print ~~the~~ this statement in python?
He said, "I want to eat an apple". we will definitely use single quotes for our convenience

$\Rightarrow$ `print('He said, "I want to eat an apple",')`

Multiline strings

If our strings has multiple lines, we can create them like this:

$\Rightarrow$ `a = '''lores ipsum may not
be same as
you will be'''`
`print(a)`

Accessing Character of a string

In Python, string is like an array of characters. We can access parts of string by using its index which starts from 0. Square brackets can be used to access elements of the string

$\Rightarrow$ `print(name[0])`
`print(name[1])`

Looping through the string

We can loop through string using a for loop like this:

$\Rightarrow$ `for character in name:`
`print(character)`

Above code print all the characters in the string name one by one!

Sequence $\Rightarrow$ `0` `R`
`1` `A`
`2` `S`
`3` `U`

→ String Slicing & operations on string
 Length of a string
 we can find the length of a string using len() function

Exmple

```
fruit = "Mango"
len1 = len(fruit)
print("Mango is a", len1, "letter word.")
```

0 ⇒ Mango is a 5 letter word.

→ String as an array
 A string is essentially a sequence of characters also called an array. Thus we can access the elements of this array.

c ⇒ pie = "ApplePie"
 print(pie[:5])
 print(pie[6])
 # return character at specified index

0 → Apple

i

Note: This method of specifying the start and end index to specify a part of a string is called slicing

Slicing Exmple

output ⇒

c ⇒ pie = "ApplePie"
 print(pie[:5]) # Slicing from start
 print(pie[5:]) # Slicing till end
 print(pie[2:6]) # Slicing in between
 print(pie[-8:]) # Slicing using negative index

Apple
 Pie
 Piep

ApplePie

Loop through a string
String are arrays are literally. Thus we can loop through string.

Ex alphabets = "ABCDE"

for i in alphabets:

print (i)

0 ⇒ A
1 B
2 C
3 D
4 E

Strings

Strings are immutable

a = !!! Ramesh !!! Ramesh

→ upper(), lower()

The upper() method converts a string to upper case.

The lower() method converts a string to lower case.

→ # The rstrip() removes any trailing characters.

print (a.rstrip("!!"))

→ The replace() method replace all occurrences of a string with another string.

print (a.replace("Ram", "Sur"))

→ The split() method splits the given string at the specified instance and returns the separated strings as list items.

print (a.split(" "))

(!!!)

(RAN)

(!!!)

→ The capitalize() method turns only the first character of the string to uppercase and the rest other characters of the string are turned to lowercase. The strings has no effect if the first character is already uppercase.

```
blogHeading1 = "introduction to tate"
print (blogHeading1.capitalize())
```

Introduction to tate.

→ The center() method aligns the strings to the center as per the parameter given by the user.

Who is Better \$\$\$.

We can also provide padding character. It will fill the rest of character provided by the user. Print (-, -, (40, ".")

..... Who is - - - - -

→ The count() method returns the number of items the given value has occurred within the given string

```
print (a.count ("Ramesh"))
```

→ The endswith() method checks if the strings ends with given value. If yes then return True, else return False.

```
p (str6.endswith ("$$$"))
```

True

We can even also check for a value in between the string by providing start and end index positions.

```
p (str6.endswith ("is", 2, 6))
```

True # ho is

⇒ The `find()` method searches for the first occurrence of the given value and returns the index where it is present.

If given value is absent from the string then return -1.

```
print (str6.find ("who")) # 1
```

```
print (str6.find ("Tate")) # -1
```

⇒ The `index()` method searches for the first occurrence of the given value and returns the index where it is present.

If given value is absent from the string then raise an exception.

```
.index ("Better")) # 7
```

⇒ The `isalnum()` method returns True only if the entire string only consists of A-Z, a-z, 0-9. If any other characters or punctuations are present, then it returns False.

```
p.isalnum()
```

```
# T or F
```

⇒ The `isalpha()` method returns True only if the entire string only consists of A-Z, a-z. If any other character or punctuations numbers (0-9) are present, then it returns False.

```
print (str6.isalpha())
```

```
T F
```

⇒ The `islower()` method returns True if all the characters in the string are lower case, else it returns False.

```
.islower())
```

```
T F
```


- ⇒ The `isprintable()` method returns True if all the values within the given string are printable, if not, then return False.
• `isprintable()` T/F
- The `isspace()` method returns True only and only if the string contains white spaces, else returns False.
- The `istitle()` returns True only if the first letter of each word of the string is capitalized else it returns False.
- The `isupper()` method returns True if all the characters in the string are upper case, else it returns False.
- The `startswith()` method checks if the a given value. If yes then returns True, else return False."
- The `Swapcase()` method changes the characters casing of the string. Upper case are converted to lower case and lower case to upper case
- The `title()` method capitalizes each letter of the word within the string.

I - My name is nassh

O - My NAME is Nash

Strings

- 1) Upper(), lower()
- 2) rstrip()
- 3) replace(), split()
- 4) capitalized(), center(), count()
- 5) ends with(), find(), index()
- 6) isalnum(), isalpha(), islower()
- 7) ^{ab}isprint(), istitle(), isupper(), is space()
- 8) start with(), end with(), swap case()

⇒ if else statements

Sometimes the programmer needs to check the evaluation of certain expression(s). whether the expression(s) evaluate to True or False. If the expression evaluates to False, then the program execution follows a different path than it would have if the expression had evaluated to True.

Based on this, the conditional statements are further classified into following types:

- if
- if - else
- if - else - elif
- nested if - else - elif

An if else statement evaluates like this:

if the expression evaluates True:

Execute the block of code inside if statement. After execution returns to the code out of the if else block.

if the expression evaluates ~~like this~~ False:

~~if the expression~~

Execute the block of code inside else statement. After execution returns to the code out of the if else block.

Example: applePrice = 210

budget = 200

if (applePrice <= budget):

print("Alexa, add 1 Kg Apples to the cart.")

else:

print("Alexa, do not add Apples to the cart.")

⇒ Alexa, do not add Apples to the cart.

⇒ elif Statements

Sometimes, the programmer may want to evaluate more than one condition, this can be done using an elif statement.

Working of an elif statement

Execute the block of code inside if statement if the initial expression evaluates to True. After execution returns to the code out of the if block

Execute the block of code inside the first elif statement if the expression inside it evaluates True. After execution returns to the code out of the if block.

Execute the block of code inside the second elif statement if the expression inside it evaluates True. After execution returns to the code out of the if block.

⋮

Execute the block of code inside the nth elif statement if the expression inside ~~the~~ it evaluate True. After execution return to the code out of the if block.

Execute the block of code inside else statement if none of the expression evaluates to True. After execution returns to the code out of the if block.

Example: num = 0

```
if (num < 0):
```

```
    print("Number is negative.")
```

```
elif (num == 0):
```

```
    { print("Number is Zero.")
```

```
else:
```

```
    print("Number is positive.")
```

0 $\Rightarrow$ Number is Zero.

⇒ Nested if statements

We can use if, if-else, elif statement inside other if statement as well.

Exp

```
num = 18
```

```
if (num < 0):
```

```
    print("Number is negative.")
```

```
elif (num > 0):
```

```
    if (num <= 10):
```

```
        print("Number is between 1-10")
```

```
    elif (num > 10 and num <= 20):
```

```
        print("Number is between 11-20")
```

```
    else:
```

```
        print("Number is greater than 20")
```

```
else:
```

```
    print("Number is zero")
```

0 ⇒ Number is between 11-20.

→ Create a python program capable of greeting you with Good Morning, Good Afternoon and Good Evening. Your program should use time module to get the current hour. Here is a sample program and documentation link for your

Match Case Statement

```
x = int(input("Enter the value of x: "))
```

```
# x is variable to match
```

```
match x:
```

```
    # if x is 0
```

```
    case 0:
```

```
        print("x is Zero")
```

```
    # case with if-condition
```

```
    case 4:
```

```
        print("case is 4")
```

```
    case - if  $x \neq 90$ :
```

```
        print(x, "is not 90")
```

```
    case - if  $x \neq 80$ :
```

```
        print(x, "is not 80")
```

```
    case - :
```

```
        print(x)
```

```
# default case (will only be matched if the  
# above cases were not matched)
```


→ Day of the week finder

```
print("Day of the Week Finder")
```

```
day-number = int(input("Enter a number (1-7):"))
```

```
match day-number:
```

```
    case 1:
```

```
        print("Monday")
```

```
    case 2:
```

```
        print("Tuesday")
```

```
    case 3:
```

```
        print("Wednesday")
```

```
    case 4:
```

```
        print("Thursday")
```

```
    case 5:
```

```
        print("Friday")
```

```
    case 6:
```

```
        print("Saturday")
```

```
    case 7:
```

```
        print("Sunday")
```

```
    case _:
```

```
        print("Invalid number! Enter 1 to 7.")
```


⇒ Goodmorning.py

```
import time
```

```
# Get the current hour (0 to 23)
```

```
current_hour = time.localtime().tm_hour
```

```
# Greet based on the hour
```

```
if 5 <= current_hour < 12:
```

```
    print("Good Morning!")
```

```
elif 12 <= current_hour < 17:
```

```
    print("Good Afternoon!")
```

```
elif 17 <= current_hour < 21:
```

```
    print("Good Evening!")
```

```
else:
```

```
    print("Hello! It's quite late. Take a Rest!")
```

```
# Optionally show the current time as well
```

```
print(f"The current time is: {time.strftime('%I:%M %p')}")
```

Output → Good Afternoon!

The current time is: 4:28 PM

→ For Loop loop in python

Sometimes a programmer wants to execute a group of statements a certain number of times. This can be done using loop. Based on this loops are further classified into following types; for loop, while loop, nested loops.

→ For Loop

for loops can iterate over a sequence of iterable objects in python. Iterating over a sequence is nothing but iterating over strings, lists, tuples, sets and dictionaries.

→ Examples: iterating over a string:

name = 'Ramesh'

for i in name: ⁰ → R, a, m, e, s, h,

print(i, end=" ") ⁰ → R a m e s h

print(" This is ^ special") ⁰ → Ram This is special esh

Ex: iterating over a list:

colors = ["Red", "Green", "Blue", "Yellow"]

for x in colors:

print(x)

⁰ → Red Green Blue Yellow

take
both same

for i in color:

print(i)

⁰ → Red Red

Green Green

Similarly, we can use loops for lists, sets and dictionaries.

range():

What if we do not want to iterate over a sequence? What if we want to use for loop for a specific number of times?

Here, we can use the range() function.

Ex: for K in range(5): → 0 1 2 3 4

Stop ←
| print(K)
| (K+1) → 1 2 3 4 5
| (K+2) → 2 3 4 5

for K in range(1, 9): → 1 2 3 4 5 6 7 8

print(K) → 1 2 3 4 5 6 7 8 9
(K+1) ←

Start ←

for K in range(1, 20001):

print(K)

Stop ←

Explore ^{3rd} step range() by own ⇔

Python while loop

As the name suggests, while loops execute statements while the condition becomes False, the interpreter comes out of the while loop.

Example: count = 5

while (count > 0):

print(count)

count = count - 1

→ + Infinite Loop

Output ⇒ 5 4 3 2 1

Ex: i = 0 (i <= 3):

while (i < 3):

print(i)

i = i + 1

Output ⇒ 0 1 2

print("Done with the Loop")

(0 1 2 3)

for i in range(3):

print(i) → 0

1
2


```

Ex: i = int(input("Enter the number: "))
while (i <= 38):
    i = int(input("Enter the number: "))
    print(i)
    print("Done with the loop")

```

→ Enter the num: 8
 9
 34
 35
 98
 Done with the loop

Here, the count variable is set to 5 which decrements after each iteration. Depending upon the while loop condition, we need to either increment or decrement the counter variable (the variable count, in our case) or the loop will continue forever.

Else with while loop

We can even use the else statement with the while loop. Essentially what the else statement does is that as soon as the while loop condition becomes False, the interpreter comes out of the while loop and the else statement is executed.

Exm: x = 5	count = 5
while (x > 0)	while (count > 0):
print(x)	print(count)
x = x - 1	count = count - 1
else:	else:
print("counter is 0")	print("I am inside else")

→ 5 4 3 2 1

counter is 0

⇒ 5 4 3 2 1 1

I am inside else

Do-while loop in python

do...while is a loop in which a set of instruction will execute at least once (irrespective of the condition) and then the repetition of loop's body will depend on the condition passed at the end of the while loop. It is also known as an exit-controlled loop.

How to emulate do while loop in python?

To create a do while loop in Python, you need to modify the while loop a bit in order to get similar behavior to a do while loop.

The most common technique to emulate a do-while loop in python is to use an infinite while loop with a break statement wrapped in an if statement that checks a given condition and breaks the iteration if that condition becomes true:

Ex: while True:

```
    number = int(input("Enter a positive number:"))
```

```
    print(number)
```

```
    if not number > 0:
```

```
        break
```

→ Enter a positive number: 1

1

Enter a positive number: 4

4

Enter a positive number: -1

-1

Explanation

This loop uses True as its formal condition. This ^{turns} tricks the loop into an infinite loop. Before the conditional statement, the loop runs all the required processing and updates the breaking condition. If this condition evaluates to true, then the break statement breaks out of the loop, and the program execution continues its normal path.

Break statement

The break statement enables a program to skip over a part of the code. A break statement terminates the very loop it lies within.

ex: for i in range(1, 101, 1):

 print(i, end=" ")

 if (i == 50):

 break

 else:

 print("Mississippi")

 print("Thank you")

1 Missi

2 —

3 —

4 —

...

50 —

Ex: for i in range(12):

 if (i == 10):

 break

 print("5 x", i+1, "=", 5 * (i+1))

print("Loop ko chodkar nikal gaya")

⇒ 5 x 1 = 5

5 x 2 = 10

=====

5 x 10 = 50

Loop ko chodkar nikal gaya

} → Break means leave the loop.
 [→ Continue means leave the iteration.

Continue Statement

The continue statement skips the rest of the loop statement and causes the next iteration to occur.

```

ex: for i in [2, 3, 4, 6, 8, 0]:
    if (i % 2 != 0):
        continue
    print(i)
  
```

⇒ 2
4
6
8
0

```

Ex: for i in range(12):
    if (i == 10):
        print("Skip the iteration")
        continue
    print("5x", i, "=", 5 * i)
  
```

5x0 = 0
 5x1 = 5
 ⋮
 5x9 = 45
 Skip the iteration
 5x11 = 55

* Do while Loop Emulate

```

i = 0
while True:
    print(i)
    i = i + 1
    if (i % 100 == 0):
        break
  
```

1 2 3 4 5 6 7 ... 99

~~## loop practice~~

Python Functions

A function is a block of code that performs a specific task whenever it is called. In bigger programs, where we have large amounts of code, it is advisable to create or use existing functions that make the program flow organized and neat.

There are two types of functions:

1. Built-in functions
2. User-defined functions

Built-in functions:

These functions are defined and pre-coded in python. Some examples of built-in functions are as follows:

min(), max(), len(), sum(), type(), range(), dict(), list(), set(), print(), etc.

User defined functions:

We can create functions to perform specific tasks as per our needs. Such functions are called user-defined functions.

Syntax:

```
def function_name(parameters):  
    pass
```

code and statements

- Create a function using the def keyword, followed by a function name, followed by a parenthesis (()) and a colon (:).
- Any parameters and arguments should be placed within the parentheses.
- Rules to naming functions are similar to the that of naming variables.

- Any statements and other code within the function should be indented.

calling a function:

We call a function by giving the function name, followed by parameters (if any) in the parenthesis.

Example:

```
def name(fname, lname):  
    print("Hello," fname, lname)
```

```
name("Sam", "Wilson")
```

⇒ Hello, Sam Wilson

⇒ Code

```
def calculateGmean(a,b):
```

```
    mean = (a*b) / (a+b)
```

```
    print(mean)
```

```
def isGreater(a,b):
```

```
    if (a>b):
```

```
        print("First number is greater")
```

```
    else:
```

```
        print("Second number is greater or equal")
```

```
def isLesser(a,b):
```

```
    pass
```


a = 9

b = 8

isGreater(a, b)

calculateGmean(a, b)

gmean1 = (a*b)/(a+b)

print(gmean1)

c = 8

d = 14

~~if (c > d):~~ isGreater(c, d)

gmean2 = (c*d)/(c+d)

print(gmean2)

calculateGmean(c, d)

⇒ First number is greater

4.23529...

Second number is greater or equal

7.21951...

→ Function Arguments and return statement

There are four types of arguments that we can provide in a function:

- Default Arguments
- Keyword Arguments
- Variable length Arguments
- Required Arguments

Default arguments:

We can provide a default value while creating a function.

This way the function assumes a default value even if a value is not provided in the function call for that argument.

Example:

```
def name(fname, mname = "Jhon", lname = "Watson"):  
    print("Hello, ", fname, mname, lname)
```

name("Amy")

⇒ Hello, Amy Jhon Watson

Keyword arguments:

We can provide arguments with key = value, this way the interpreter recognizes the arguments by the parameter name. Hence the order in which the arguments are passed does not matter.

Example:

```
def name(fname, mname, lname):  
    print("Hello, ", fname, mname, lname)
```

name(mname = "Peter", lname = "Wesker", fname = "Jade")

⇒ Hello, Jade Peter Wesker

Required arguments:

In case we don't pass the arguments with a key = value syntax then it is necessary to pass the arguments in the correct positional order and the number of arguments passed should match with actual function definition.

Example 1: when number of arguments passed does not match to the actual function definition.


```
def name (fname, mname, lname):  
    print ("Hello, ", fname, mname, lname)
```

```
name ("Peter", "Quill")
```

⇒ name ("Peter", "Quill")

TypeError: name() missing 1 required positional argument: 'lname'

Ex 2: when number of arguments passed matches to the actual function definition.

```
⇒ def name (fname, mname, lname):  
    print ("Hello, ", fname, mname, lname)
```

```
name ("Peter", "Ego", "Quill")
```

⇒ Hello, Peter Ego Quill

Variable - length arguments:

Sometimes we may need to pass more arguments than those defined in the actual function. This can be done using variable length arguments.

There are two ways to achieve this:

Arbitrary Arguments:

While creating a function, pass a* before the parameter name while defining the function. The function accesses the arguments by processing them in the form of tuple.

Example:

```
def name (*name):
```

```
    print ("Hello, ", name[0], name[1], name[2])
```

```
name ("James", "Buchanan", "Barnes")
```

→ Hello, James Buchanan Barnes

Keyword Arbitrary Arguments:

While creating a functions, pass a * before the parameter name while defining the function. The function accesses the arguments by processing them in the form of dictionary.

Ex:

```
def name (**name):
```

```
    print ("Hello, ", name["fname"], name["mname"],  
          name["lname"])
```

```
name(mname="Buchanan", lname="Barnes", fname="James")
```

⇒ Hello, James Buchanan Barnes

return Statement

The return statement is used to return the value of the expression back to the calling function.

Ex.

```
def name(fname, mname, lname):
```

```
    return "Hello, " + fname + " " + mname + " " +  
    lname
```

```
print (name ("James", "Buchanan", "Barnes"))
```

⇒ Hello, James Buchanan Barnes

Python Lists

- Lists are ordered collection of data items
- They store multiple items in a single variable.
- List items are separated by commas and enclosed within square brackets[].
- Lists are changeable meaning we can alter them after creation.

Ex. 1st 1 = [1, 2, 2, 3, 5, 4, 6]

1st 2 = ["Red", "Green", "Blue"]

print (1st1)

print (1st2)

⇒ [1, 2, 2, 3, 5, 4, 6]

['Red', 'Green', 'Blue']

Ex. 2 details = ["Abhijeet", 18, "FYBSCIT", 9.8]

print (details)

⇒ ['Abhijeet', 18, 'FYBSCIT', 9.8]

As we can see, a single list can contain items of different data types.

Code ⇒ marks = [3, 5, 6, "Harry", True]

print (marks)

print (type (marks))

print (marks [0])

print (marks [1])

print (marks [2])

print (marks [3])

print (marks [4])

print (marks [5])

Syntax:

listName[start : end : jumpIndex]

Note: Jump Index is optional. we will see this in later examples.

Example: printing elements within a particular range:

```
animals = ["cat", "dog", "bat", "mouse", "pig", "horse",  
           "donkey", "goat", "cow"]
```

```
print(animals[3:7]) # using positive indexes
```

```
print(animals[-1:-2]) # using negative indexes
```

⇒ ['mouse', 'pig', 'horse', 'donkey']

['bat', 'mouse', 'pig', 'horse', 'donkey']

Here we provide index of the element from where we want to start and the index of the element till which we want to print the values.

Note: The elements of the end index provided will not be included.

```
code c ⇒ print(marks)  
          print(marks[1:-1])  
          print(marks[1:4:2])  
          ✓
```

It will print till 4th range by jumping 2 and 2

1 1 ✓ 1 1 ✓
1 2 3 4

till 4th


```

colors = ["Red", "Green", "Blue", "Yellow", "Green"]
#           [-5]   [-4]   [-3]   [-2]   [-1]

```

```
print (colors [-1])
```

```
print (colors [-3])
```

```
print (colors [-5])
```

Output: Green Blue Red

⇒ Code Continue:

```

print (marks [-3]) # Negative index
print (marks [len (marks) - 3]) # Positive index
print (marks [5 - 3]) # Positive index
print (marks [2]) # Positive index

```

Check whether an item is present in the list?

We can check if a given item is present in the list. This is done using the `in` keyword

```

colors = ["Red", "Green", "Blue", "Yellow", "Green"]

```

```

if "Yellow" in colors:
    print ("Yellow is present.")

```

```
else:
```

```
    print ("Yellow is absent.")
```

⇒ Yellow is present

Range of Index:

You can print a range of list items by specifying where you want to start, where do you want and if you to skip elements in between the range:

List Index

Each item/element in a list has its own unique index. This index can be used to access any particular item from the list. The first item has index [0], second item has index [1], third item has index [2] and so on.

Example:

colors = ["Red", "Green", "Blue", "Yellow", "Green"]
 [0] [1] [2] [3] [4]

Accessing list items

We can access list items by using its index with the square bracket syntax []. For example colors [0] will give "Red", color [1] will give "Green" and so on...

Positive Indexing:

As we have seen that list items have index, as such we can access items using these indexes.

Example: colors = ["Red", "Green", "Blue", "Yellow", "Green"]
 # [0] [1] [2] [3] [4]

print(colors[2])

print(colors[4])

print(colors[0])

⇒ Blue Green Red

→ Negative Indexing:

Similar to positive indexing negative indexing is also used to access ~~last~~ items, but from the end of the list. The last item has index [-1], second last item has index [-2], third item has index [-3] and so on.


```
print(animals[4:]) #pi  
print(animals[-4:]) #Ni
```

When no end index is provided, the interpreter prints all the values till the end.

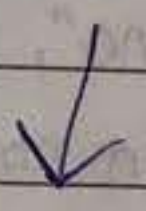
```
print(animals[:4]) #pi  
print(animals[:5]) #Ni
```

Same as Here.

Printing alternate values

```
print(animals[::2]) #PI  
print(animals[-8:-1:2]) #NI
```

: All values by jumping 2.



-8 till -1 by jumping 2.

→ print 3 consecutive value range

```
(animals[1:8:3])
```

Here, jump index is 3. Hence it prints every 3rd element within given index.

List Comprehension

List comprehensions are used for creating new lists from other iterables like lists, tuples, dictionaries, sets, and even in array in arrays and strings.

Ex 1: Accepts items with the small letter "o" in the new list

```
names = ["Milo", "Sarah", "Bruno", "Anastasia", "Rosa"]  
nameswith_o = [item for item in names if "o" in item]  
print (nameswith_o)
```

⇒ ['Milo', 'Bruno', 'Anastasia', 'Rosa']

Ex 2: Accepts items which have more than 4 letters

```
name = ["Milo", "Sarah", "Bruno", "Anastasia", "Rosa"]  
namewith_o = [item for item in name if (len(item) > 4)]  
print (nameswith_o)
```

⇒ ['Sarah', 'Bruno', 'Anastasia']

Python List Methods

→ list.sort()

This method sorts the list in ascending order. The original list is updated

```
num = [8, 7, 2, 1, 9, 6]
```

```
num.sort()
```

```
print (num)
```

⇒ [1, 2, 6, 7, 8, 9]

What if you want to print the list in descending order?
we must give (reverse=True) as a parameter in the sort method.

l.sort(reverse=True)

It will [↑] print your list in descending order

→ reverse() method

This method reverses the order of the list.

num = [1, 2, 3, 4, 5]

num.reverse()

print(num)

⇒ [5, 4, 3, 2, 1]

⇒ index()

This method returns the index of the first occurrence of the list item.

Ex: num = [1, 2, 3, 4, 5, 6, 7]

print(num.index(3))

(3) ⇒ [2]

⇒ [6]

→ count()

Returns the count of the number of items with the given value.

Ex:

print(num.count(5))

print(l num).

1
⇒ [1]

⇒ `copy()`
Returns copy of the list. This can be done to perform operation on the list without modifying the original list.

```
num = [1, 2, 3, 4, 5, 6, 7]
newnum = num.copy()
print(num)
print(newnum)
```

⇒ `[1 2 3 4 5 6 7]`
`[1 2 3 4 5 6 7]`

→ `append()`:

This method appends items to the end of the existing list.

```
num = [1, 2, 3, 4, 5, 6, 7]
num.append("9")
print(num)
```

→ `[1 2 3 4 5 6 7 9]`

→ `insert()`:

This method inserts an item at the given index. User has to specify index and the item to be inserted within the `insert()` method.

```
Ex: num = [1, , , , 7]
num.insert(1, "5")
print(num)
```

⇒ `[1, 5, , , , 7]`

→ extend():

This method adds an entire list or any other collection datatype (set, tuple, dictionary) to the existing list.

Ex:1 # add a list to a list

```
num = [ 1 - 4 ]
```

```
numbers = [ 5 - 7 ]
```

```
→ num.extend(numbers)
   print(num)
```

⇒ [1 2 3 4 5 6 7]

Concatenating two lists

You can simply concatenate two lists to join two lists

```
Ex: num = [ 1 - 5 ]
```

```
numbers = [ 6 - 9 ]
```

```
print(num + numbers)
```

[1 - 9]

~~remove~~.remove("mango")

```
popped_item = fruits.pop(2).pop()
```

↓

it will remove the 0/2nd item of in the list.


```
num = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
print(num)
```

```
num.append(7)
```

```
num.sort(reverse=True)
```

```
num.reverse()
```

```
num print(num.index(1))
```

```
print(num.count(1))
```

```
num.insert(1, 899)
```

```
num.extend
```

```
print(num)
```

```
num.sort()
```

```
num.copy()
```

```
print(num + numbers)
```

```
num.remove(899)
```

```
popped-item = num.pop(2)
```

Python Tuples

Tuples are ordered collection of data items. They store multiples items in a single variable. Tuple items are separated by commas and enclosed within round brackets (). Tuples are (UNCHANGABLE) meaning we can not alter them after creation.

```
 tup = (1, 2, 3)
```

```
 tup1 = (47, Red, Colors, Green)
```

```
 print(tup)
```

```
 print(tup1)
```

⇒ (1, 2, 3)

(Red, Colors, Green)

Ex: 2

```
details = ("Ram", 17, "FYOOP", 9.8)
print(details)
```

⇒ ("Ram", 17, "FYOOP", 9.8)

⇒ Tuple indexes

```
print(details[1])
```

⇒ 17

How negative indexing works is

first len(num) (1, 2, 3, 4, 5, 6, 7)
⇒ 7 is len

(num[-1]) ⇒ 7 - 1 = 6

how it works → (7)

→ Check for item:

We can check if a given item is present in the tuple.

This is done using the in keyword.

Same we can do with lists also.

```
if "Ram" in details:
```

```
    print("Yes")
```

```
else:
```

```
    print("No")
```

Range of index: Tuples

Tuple[start : end : jumpIndex]

⇒ Printing Elements

```
print(animals[3:7])  
[-7:-2]
```

All types of slicing done in lists can be done in Tuples as well.

Note Tuples are immutable it cannot be changed or create new one

⇒ Manipulating Tuples

Tuples are immutable, hence if you want to add, remove or change tuple items, then first you must convert the tuple to a list. Then perform operation on that list and convert it back to tuple.

[Example:

```
countries = ("Spain", "Italy", "India", "England", "Germany")
```

```
temp = list(countries)
```

```
temp.append("Russia") # add item
```

```
temp.pop(3) # remove item
```

```
temp[2] = "Finland" # change item
```

```
countries = tuple(temp)
```

```
print(countries)
```

⇒ ('Spain', 'Italy', 'Finland', 'Germany', 'Russia')

Syntax: Tuple to list first

```
temp = list(countries)
```

↓

Tuple → list

Syntax: Then the list was created by tuple now again create it to a tuple by list

```
countries = tuple(temp)
```

↓

list to the tuple.

Then, we convert the tuple to a list, manipulate item of the list using list methods, then convert list back to a tuple.

⇒ However, we can directly [concatenate] two tuples without converting them to list.

Ex:

```
countries 1 = ("Pakistan", "Afghanistan",
```

```
countries 2 = (
```

```
South East Asia = countries + countries 2
```

```
print (South East Asia)
```

Tuples Methods

⇒ count() Method

The count() Method of Tuple returns the number of times the given element appears in the tuple.

```
tuple.count(element)
```

```
res = Tuple 1.count(3)
```

→ index() method

The Index() method returns the first occurrence of the given element from the tuple.

len tuple

```
res = len (tuple 1)
```

```
tuple.index(3, start, end)
```

⇒ [10]

```
index(3, 4, 8)
```

first occurrence.

1. Create a program capable of displaying questions to the user like KBC.
2. Use List data type to store the questions and their correct answers
3. Display the final amount the person is taking home after playing the game.

⇒ f-strings in python.

String formatting in python

String formatting can be done in python using the format method.

```
txt = 'For only {price:2f} dollars!'
print (txt.format(price = 49))
```

f- string in python

Ex: val = 'Geeks'

```
print (f" {val} for {val} is a portal for {val}."
```

```
name = 'Tushar'
```

```
age = 23
```

```
print (f"Hello, My name is {name} and I'm {age} years old.")
```

In the above code, we have used the f string to format the string.

We can use it in a single statement as well.

Ex: `print(f" {2 * 30} ")`

0 ⇒ 60

Code :

```
letter = "Hey my name is {1} and I am from {0}"
country = "India"
name = "Harry"
```

```
print (letter.format(country, name))
print (f"Hey my name is {name} and I am from {country}")
```

```
print (f"Hey my name is {{name}} and I am from {{country}}")
```

```
price = 49.09999
```

```
txt = f"For only {price:2f} dollars!"
print (txt)
```

```
# print (txt.format())
```

```
print (type(f" {2 * 30}"))
```

⇒

Docstrings in python

Python Docstrings are the string literals that appear right after the definition of a function, method, class, or module.

Ex: `def square(n):`

⇒ `"""Takes in a number n, returns the square of n"""`

`print (n**2)`

`square(5)`

`print (square.__doc__)`

Docstring must be after function name only not somewhere else. otherwise it does not work.

Here,

`"""Takes in a number n, returns the square of n"""` is a docstring which will not appear in output.

⇒ 25

The arrow lines is known as Docstring not an comments.

PEP

PEP - 8 (Python Enhancement Proposal)

PEP-8

pip3 install

Open Terminal

python

import this

'Zen of Python' it prints

It print the poem and it relate to your code much.

This is a just Easter Egg in python

→ Recursion in python

Recursion is the process of defining something in terms of itself.

A physical world example would be ~~the~~ to place two parallel mirrors facing each other. Any object in between them would be reflected recursively.

Python Recursion Function

In Python, we know that a function can call other functions.

It is possible for the function to call itself. These types of construct are termed as recursive functions.

def factorial(n):

if (n==0) or (n==1):

return 1

else:

return n * factorial(n-1)

print (factorial(5))

⇒ 120

Methods of String

Methods of lists

1) upper()

2) lower()

3) strip()

4) replace()

5) split()

6) join()

7) find()

8) startswith()

9) endswith()

10) count()

11) title()

12) capitalize()

13) isdigit()

14) isalpha()

15) isalphanumeric()

16) swapcase()

1) append()

2) extend()

3) insert()

4) remove()

5) pop()

6) copy()

7) index()

8) sort()

9) reverse()

10) count()

11) copy()

Methods of Sets

1) add()

2) update()

3) remove()

4) discard()

5) pop()

6) union()

7) intersection()

8) difference()

9) Symmetric difference()

10) issubset()

11) issuperset()

12) is disjoint()

13) copy()

14) clear()

Methods of dictionaries

1) Keys()

2) values()

3) items()

4) get()

5) update()

6) pop()

7) pop items()

8) setdefault()

9) copy()

10) clear()

11) fromkeys()

Python - else in Loop

Python allows -else statement to be used in Loops
Yes you can use -else statement in Loop not only with if statement but with Loop also - Remember it.

{ Error handling } PYTHON
{ Exception handling }

Exception handling is the process of responding to unwanted or unexpected events when a computer program runs, Exception handling deals with these events to avoid the program or system crashing and without this process, exception would disrupt the normal operation of a program.

Python has many built-in exceptions that are raised when your program encounters an error (something in the program goes wrong).

When these exceptions occur, the Python interpreter stops the current process and passes it to the calling process until it is handled, ~~the program will~~ if not handled, the program will crash.

Python

try ... except
try ... except blocks are used in Python to handle errors and exceptions. The code in try block runs when there is no error. If the try block catches the error, then the except block is executed.


```
try:
    # statements which could generate
    # exception
except:
    # Solution of generated exception
```

⇒ Enumerate Function in Python

The enumerate function is a built-in function in Python that allows you to loop over a sequence (such as a list, tuple, or string) and get the index and value of each element in the sequence at the same time. Here's basic example of how it works.

```
fruits = ['apple', 'banana', 'mango']
for index, fruit in enumerate(fruits):
    print(index, fruit)
```

↳ # Loop over a list and print the index and value of each element.

```
⇒ 0 apple
   1 banana
   2 mango
```